EIU
TRƯỜNG ĐẠI HỌC QUỐC TẾ MIỀN ĐÔNG
EASTERN INTERNATIONAL UNIVERSITY

FINAL EXAM PAPER
Quarter: 1 - Academic year: 2025 – 2026
School of Computing and Information Technology

Course code: CSW 306 | **Exam code: 01**
Course name: Backend Development | **Duration: 120'**
Type of Exam: Open book / Practice on Computer | **Maximum mark: 100**

Instructions:

- *Non-programming calculator: Yes ☐ | No ☐*
- *This paper consists of 3 questions. This paper carries 100 marks:*
  Question 1 (30 marks).
  Question 2 (30 marks).
  Question 3 (40 marks).

Students must create the project using the naming convention:

Format: DateNow (DD-MM-YYYY)_BE_YourName

Example: 19122025_BE_NguyenXuanCuong

Students zip the project source code and include the database script or backup file in one compressed file for submission.

Students may refer to the .NET documentation to design and develop ASP.NET Core API web applications using the following:

1. https://learn.microsoft.com/en-us/aspnet/core/getting-started/?view=aspnetcore-8.0.
2. Print or pdf documents available.

FRUITS STORE MANAGEMENT SOFTWARE

Company ProMaxSuperUltraBeauty wants to build an internal Web API system to manage their fruit store and employees. You are required to develop a Web API to fulfill the following requirements.

Question 1: Database, Model and DbContext Design (30 marks)

Task: Design the core database and model for the Fruit Store Management System.

1. Create the following classes with relationships:
   - Fruit **belongs** to one Category
   - Category **has many** Fruits
   - User **has one** Role (Admin or User) – stored as a property in User
   - Order **has many** OrderItems

- OrderItem **belongs to one** Order and one Fruit

2. Add **Data Annotations / Validation**:
   - Fruit: Name (required, max 100 chars), Price (required, positive)
   - Category: Name (required, max 50 chars)
   - User: FullName (required, max 200 chars), Email (required, valid email), Role (required, enum or string "Admin" / "User")
   - Order: OrderDate (cannot be in future)
3. Create a class **AppDbContext** that declares:
   - DbSet<Fruit> Fruits
   - DbSet<Category> Categories
   - DbSet<User> Users
   - DbSet<Order> Orders
   - DbSet<OrderItem> OrderItems

Note: Tables should be created in the database corresponding to the classes.

Table: User

Field	**Type**	**Constraints / Notes**
Id	INT	PK, Identity / Auto-increment
FullName	NVARCHAR(200)	
Email	NVARCHAR(255)	NOT NULL, UNIQUE, Email format validation
Password	NVARCHAR(MAX)	NOT NULL (Optional: Hash Password for more secure)
Role	NVARCHAR(20)	NOT NULL: 'Admin' / 'User'
CreatedAt	DATETIME	Default GETDATE()

Table: Categories

Field	**Type**	**Constraints / Notes**
Id	INT	PK, Identity
Name	NVARCHAR(50)	NOT NULL, UNIQUE
Description	NVARCHAR(255)	Optional
CreatedAt	DATETIME	Default GETDATE()

Table: Fruits

Field	**Type**	**Constraints / Notes**
Id	INT	PK, Identity

Name	NVARCHAR(100)	NOT NULL
Price	DECIMAL(10,2)	NOT NULL, positive
StockQuantity	FLOAT	Default: 0
CategoryId	INT	FK → Categories(Id), NOT NULL
CreatedAt	DATETIME	Default GETDATE()

Table: Orders

Field	Type	Constraints / Notes
Id	INT	PK, Identity
UserId	INT	FK → Users(Id), NOT NULL
OrderDate	DATETIME	NOT NULL, cannot be in the future
TotalAmount	DECIMAL(10,2)	Computed or calculated
Status	NVARCHAR(20)	e.g., 'Pending', 'Completed', 'Cancelled'
CreatedAt	DATETIME	Default GETDATE()

Table: OrderItems

Field	Type	Constraints / Notes
Id	INT	PK, Identity
OrderId	INT	FK → Orders(Id), NOT NULL
FruitId	INT	FK → Fruits(Id), NOT NULL
Quantity	INT	NOT NULL, positive
UnitPrice	DECIMAL(10,2)	Price at the time of order
CreatedAt	DATETIME	Default GETDATE()

Question 2: Repository Layer (30 marks)

Task: Implement repository classes to interact with the database using **AppDbContext**.

1. Create **IFruitRepository** and **FruitRepository** (30 marks)

Include the following methods:

- Task<List<Fruit>> GetAll(); (5 marks)
- Task<Fruit?> GetById(int id); (5 marks)
- Task Add(Fruit fruit); (5 marks)
- Task Update(Fruit fruit); (5 marks)
- Task Delete(int id); (5 marks)
- Task<List<Fruit>> GetByCategoryId(int categoryId); (5 marks)

All methods must use **AppDbContext** for database operations.

Note: Repositories for **Category, User, Orders, and OrderItems** are not required, but can be implemented similarly if you choose to, following the same structure and pattern.

Question 3: ***API Controller (40 marks)***

Task: Create REST APIs for managing Fruits and Orders.

1. **FruitsController** endpoints:

Method	**Route**	**Description**
GET	/api/fruits	- Return fruits with category info (nested or flat structure accepted) - Response format for GET /api/fruits (both formats accepted): Option 1 (Nested): {"id": 1, "name": "Apple", "category": {"id": 1, "name": "Fresh Fruits"}} Option 2 (Flat): {"id": 1, "name": "Apple", "categoryId": 1, "categoryName": "Fresh Fruits"}
GET	/api/fruits/{id}	Return fruit by ID
POST	/api/fruits	Create new fruit
PUT	/api/fruits/{id}	Update fruit
DELETE	/api/fruits/{id}	Delete fruit

2. **OrdersController** endpoints:

Method	**Route**	**Description**
GET	/api/orders	List orders (include items)

POST	/api/orders	Create new order with list of order items
		POST /api/orders
		Request body example:
		{
		
		"items": [
		{"fruitId": 1, "quantity": 5},
		{"fruitId": 2, "quantity": 3}
		]
		}
		
		System automatically:
		- Extract **userId** from JWT token
		- Set **orderDate** = current server time
		- Set **status** = "Pending"
		- Calculate **unitPrice** from current **Fruit.Price**
		- Calculate **totalAmount** = sum of (**quantity** × **unitPrice**)
		- Validate: fruitId exists, quantity > 0, stock sufficient

GET	/api/orders/user/{userId}	- Get orders by userId

		- System must validate user can only view their own orders (userId in route must match userId in JWT token)

3. **AuthController** endpoints:

Method	Route	Description
POST	/api/auth/login	Login to receive a JWT token for authentication.

4. **CategoryController** endpoints:

Method	Route	Description
GET	/api/categories	Return list of all categories.

Requirements:

- Use dependency injection to inject services.
- Handle **404 Not Found** for invalid routes.
- Validate input data:
    - Use Data Annotations ([Required], [Range], [EmailAddress],...)
    - Return 400 Bad Request with error messages if validation fails
    - Check foreign key existence (e.g., CategoryId, UserId must exist)
- Configure **Security**:
    - Implement JWT-based authentication
    - Hash user passwords using BCrypt or ASP.NET Identity (Optional)
    - Authorization rules:
        - Admin: can CREATE/UPDATE/DELETE fruits
        - Admin: can view all orders (GET /api/orders)
        - Authenticated users: can create orders (POST /api/orders)
        - Users: can only view their own orders (GET /api/orders/user/{userId})
        - Public (no login): can view fruits and categories

IMPORTANT NOTES:

- You are NOT required to implement user registration functionality.

- For testing purposes, manually create test users directly in the database with the following requirements:

  - At least 1 Admin user (Role = "Admin")

  - At least 1 Normal user (Role = "User")

  - Passwords can be stored as plain text for testing (no hashing required)

- Example users:
  + Email: admin@test.com, Password: admin123, Role: Admin
  + Email: user@test.com, Password: user123, Role: User
- Include these test users in your database backup file submission.
- Complete the easier tasks first to optimize your time.

------*The End* ------

Lecturer's signature

Cuong Nguyen Xuan